Міністерство освіти і науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №3

з дисципліни «Об’єктно-орієнтоване програмування»

на тему:

«Розробка інтерфейсу користувача графічного редактора об'єктів»

Виконав:
студент групи ІМ-55
Присяжний С. О.
Номер у списку: 25
(Варіант Ж = 26)

Перевірив:
Рекечинський Д. О.

Київ 2026

Мета роботи:

Отримати стійкі практичні вміння та навички проектування графічного інтерфейсу користувача (GUI) з використанням ключових принципів об'єктно-орієнтованого програмування: інкапсуляції, абстракції типів, успадкування та динамічного поліморфізму. Реалізувати повнофункціональну панель інструментів (Toolbar) з власними іконками, інтерактивними спливаючими підказками (Tooltips), візуальною фіксацією активного стану кнопок та повною синхронізацією з підпунктами системного меню.

Завдання:

1. Розширити графічний редактор об'єктів шляхом додавання дочірнього вікна/компонента панелі інструментів (Toolbar).

2. Реалізувати для вибору типу об'єкта кнопки Toolbar з відповідними векторними іконками (Крапка, Лінія, Прямокутник, Еліпс), що дублюють пункти меню «Об'єкти».

3. Реалізувати механізм спливаючих текстових підказок (Tooltips) для кнопок панелі інструментів (аналог обробки WM_NOTIFY / TTN_NEEDTEXT).

4. Забезпечити візуальну фіксацію натиснутого стану (pressed state / toggle) активної кнопки Toolbar (аналог TB_PRESSBUTTON).

5. Забезпечити взаємну узгодженість та синхронізацію стану між панеллю інструментів, гарячими клавішами та системним меню «Об'єкти».

6. Перерахувати індивідуальні параметри варіанта за формулою Ж = Жлаб2 + 1 = 26 та оновити відповідні класи фігур і редакторів.

7. Оформити діаграму класів (Class Diagram), схему архітектури взаємодії компонентів, вихідний текст модулів, скріншоти тестування та академічні відповіді на контрольні запитання.

Розрахунок та характеристика індивідуального варіанта:

Відповідно до п. 4 методичних вказівок до лабораторної роботи №3, номер варіанта обчислюється як Ж = Жлаб2 + 1, де Жлаб2 = 25 — номер студента у списку академічного журналу. Таким чином, розрахункове значення індивідуального варіанта становить: Ж = 25 + 1 = 26.

На основі значення Ж = 26 отримано наступні параметри виконання роботи:

1. Організація масиву об'єктів Shape:

Умова вибору: якщо (Ж mod 3 = 0) — динамічний масив, інакше — статичний масив.
Кількість елементів N = Ж + 100.
Розрахунок: 26 mod 3 = 2 ≠ 0 ⇒ обирається статичний масив покажчиків pcshape[N].
Розмір буфера: N = 26 + 100 = 126 елементів.

2. «Гумовий» слід при інтерактивному введенні об'єктів:

Умова вибору: (Ж mod 4):
0 — суцільна лінія чорного кольору; 1 — суцільна лінія червоного кольору; 2 — суцільна лінія синього кольору; 3 — пунктирна лінія чорного кольору. Розрахунок: 26 mod 4 = 2 ⇒ призначається суцільна лінія синього кольору (#2563eb).

3. Різновид введення та відображення прямокутника (Rect):

• Спосіб введення (Ж mod 2): 26 mod 2 = 0 ⇒ по двох протилежних кутах.
• Відображення прямокутника (Ж mod 5): 26 mod 5 = 1 ⇒ чорний контур з кольоровим заповненням.
• Колір заповнення прямокутника (Ж mod 6): 26 mod 6 = 2 ⇒ блакитний колір (#38bdf8).

4. Різновид введення та відображення еліпса (Ellipse):

• Спосіб введення (Ж mod 2): 26 mod 2 = 0 ⇒ від центру до одного з кутів охоплюючого прямокутника.
• Відображення еліпса (Ж mod 5): 26 mod 5 = 1 ⇒ чорний контур з білим заповненням (#ffffff).

5. Позначка поточного типу об'єкта, що вводиться:

Умова вибору: (Ж mod 2):
0 — у підпунктах меню (метод OnInitMenuPopup); 1 — у заголовку вікна програми. Розрахунок: 26 mod 2 = 0 ⇒ позначка поточного типу інструмента виставляється безпосередньо у випадаючому меню «Об'єкти» (радіо-позначка checked) із синхронізацією стану затиснутої кнопки в Toolbar.

Схеми та діаграми структури програми:

На основі розробленої об'єктно-орієнтованої структури створено оновлену UML-діаграму класів з урахуванням доданого компонента ToolBar (Рис. 1), а також схему архітектури компонентів та IPC-взаємодії (Рис. 2).

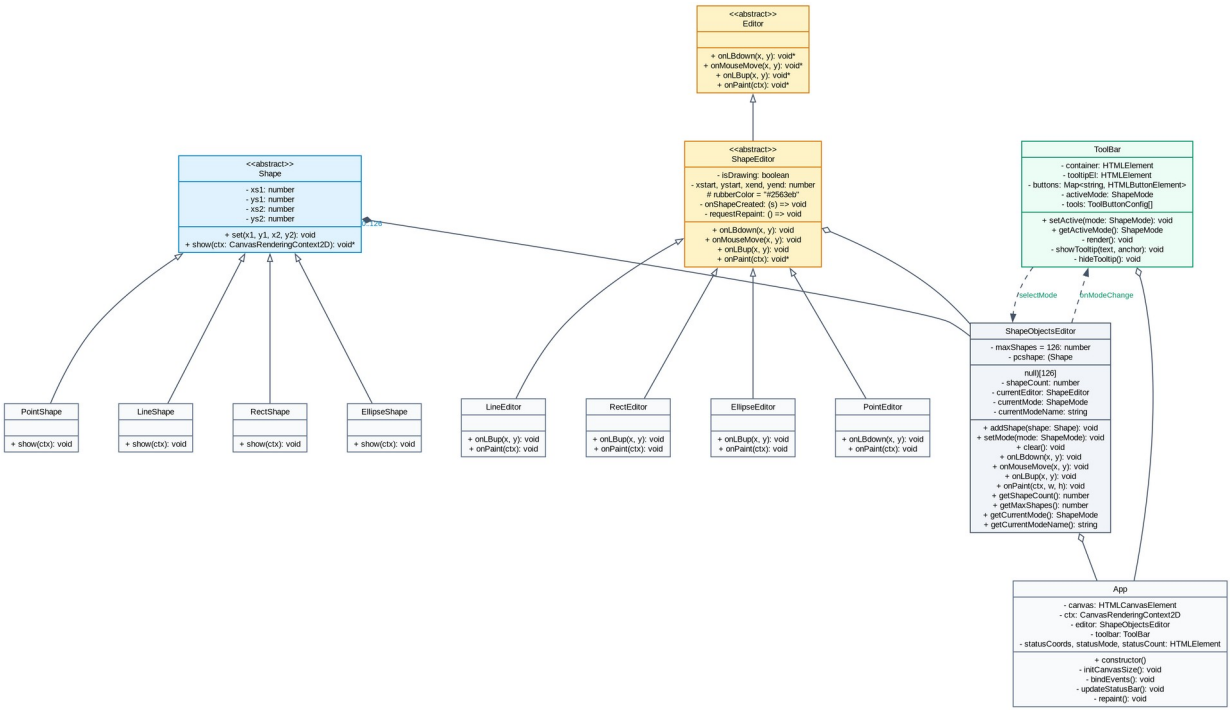


Рис. 1. UML діаграма класів графічного редактора (ієрархії Shape, Editor, координатор ShapeObjectsEditor та компонент ToolBar)

На діаграмі класів (Рис. 1) представлено ключові складові системи:
1. Ієрархія даних (Shape): базовий абстрактний клас Shape інкапсулює прямокутні координати та абстрактний метод show(ctx). Похідні класи PointShape, LineShape, RectShape (блакитна заливка) та EllipseShape (біла заливка від центру) поліморфно відмальовують фігури.
2. Ієрархія поведінки (Editor): базовий клас Editor декларує віконні методи миші. Клас ShapeEditor інкапсулює базовий синій гумовий слід (#2563eb). Похідні класи PointEditor, LineEditor, RectEditor (по 2 кутах) та EllipseEditor (від центру до кута) обробляють інтерактивні події введення.
3. Клас-координатор ShapeObjectsEditor: інкапсулює статичний масив pcshape розміром 126 елементів, делегує події поточному редактору та сповіщає зовнішні інтерфейси про зміну стану.
4. Компонент ToolBar: окремий клас інтерфейсу, що інкапсулює кнопки, векторні піктограми, активний стан (pressed state) та спливаючі підказки (tooltips).

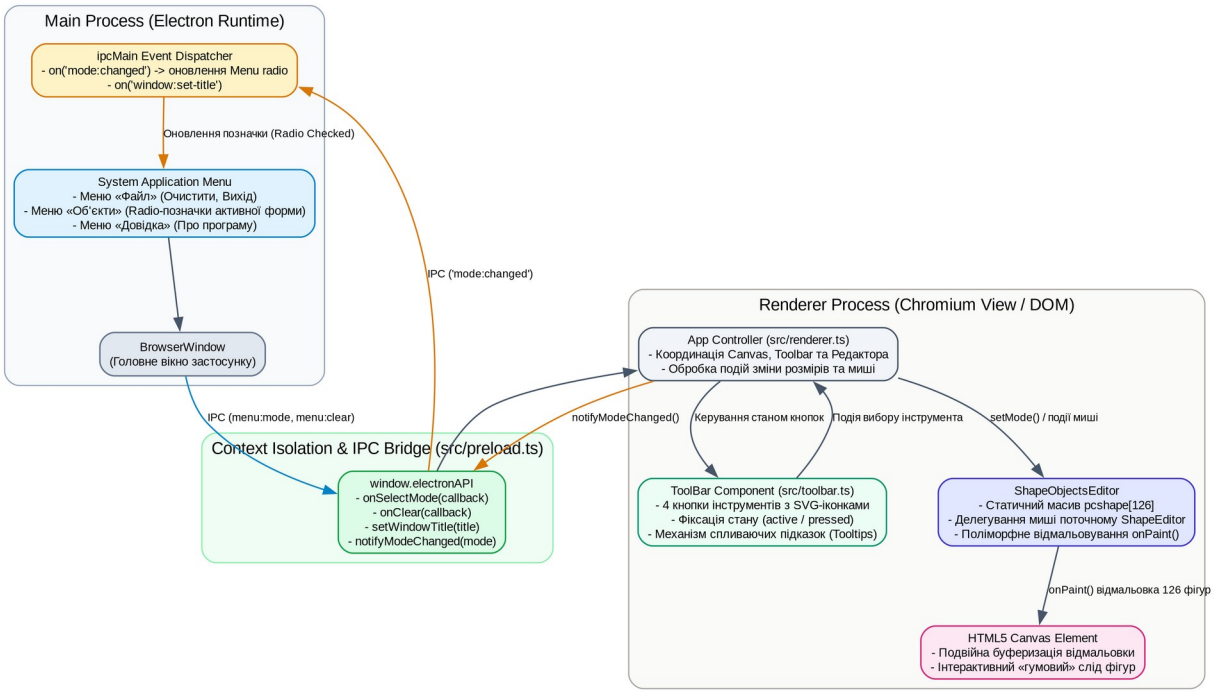


Рис. 2. Архітектура модулів та схема IPC-взаємодії системного меню, панелі інструментів і полотна

Діаграма архітектури (Рис. 2) демонструє дворівневу взаємодію: головний процес керування вікном (Main Process) із нативним системним меню, безпечний міст ізоляції контексту (Preload Bridge) та процес відображення (Renderer Process), де координуються взаємодія ToolBar, Canvas та ShapeObjectsEditor.

Вихідний текст програми:

Нижче наведено вихідні тексти ключових модулів розробленого графічного редактора. Особливу увагу приділено новому класу ToolBar, синхронізації системного меню та оновленим класам фігур.

src/toolbar.ts (Клас панелі інструментів ToolBar, керування кнопками, підказками та активним станом)

```
import type { ShapeMode } from './shape_objects_editor.ts';

export interface ToolBarCallbacks {
  onSelectMode: (mode: ShapeMode) => void;
  onClear: () => void;
}

interface ToolButtonConfig {
  id: ShapeMode | 'clear';
  name: string;
  tooltip: string;
  isAction?: boolean;
  svg: string;
}
```

```
export class ToolBar {
  private container: HTMLElement;
  private tooltipEl: HTMLElement;
  private buttons: Map<string, HTMLButtonElement> = new Map();
  private activeMode: ShapeMode = 'point';

  private readonly tools: ToolButtonConfig[] = [
    {
      id: 'point',
      name: 'Крапка',
      tooltip: 'Крапка (Ctrl+1)',
      svg: '<svg viewBox="0 0 24 24" width="18" height="18"><circle cx="12" cy="12" r="3"
fill="currentColor"/></svg>',
    },
    {
      id: 'line',
      name: 'Лінія',
      tooltip: 'Лінія (Ctrl+2)',
      svg: '<svg viewBox="0 0 24 24" width="18" height="18"><line x1="5" y1="19" x2="19" y2="5"
stroke="currentColor" stroke-width="2.5" stroke-linecap="round"/></svg>',
    },
    {
      id: 'rect',
      name: 'Прямокутник',
      tooltip: 'Прямокутник (Ctrl+3)',
      svg: '<svg viewBox="0 0 24 24" width="18" height="18"><rect x="4" y="5" width="16" height="14"
fill="#38bdf8" stroke="currentColor" stroke-width="2" rx="1"/></svg>',
    },
    {
      id: 'ellipse',
      name: 'Еліпс',
      tooltip: 'Еліпс (Ctrl+4)',
      svg: '<svg viewBox="0 0 24 24" width="18" height="18"><ellipse cx="12" cy="12" rx="9" ry="6" fill="#ffffff"
stroke="currentColor" stroke-width="2"/></svg>',
    },
    {
      id: 'clear',
      name: 'Очистити',
      tooltip: 'Очистити полотно (Ctrl+N)',
      isAction: true,
      svg: '<svg viewBox="0 0 24 24" width="18" height="18"><path d="M4 7h16M10 11v6M14 11v6M5 7l1 12a2
2 0 0 0 2 2h8a2 2 0 0 0 2-2l1-12M9 7V4a1 1 0 0 1 1-1h4a1 1 0 0 1 1 1v3" fill="none" stroke="currentColor"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"/></svg>',
    },
  ];

  constructor(
    containerSelector: string,
```

```
  tooltipSelector: string,
  private readonly callbacks: ToolBarCallbacks,
) {
  this.container = document.querySelector<HTMLElement>(containerSelector)!;
  this.tooltipEl = document.querySelector<HTMLElement>(tooltipSelector)!;

  this.render();
  this.setActive('point');
}

private render(): void {
  this.container.innerHTML = '';

  for (const config of this.tools) {
    if (config.isAction) {
      const sep = document.createElement('div');
      sep.className = 'toolbar-separator';
      this.container.appendChild(sep);
    }

    const btn = document.createElement('button');
    btn.type = 'button';
    btn.className = 'toolbar-btn';
    btn.dataset.id = config.id;
    btn.setAttribute('aria-label', config.name);
    btn.innerHTML = config.svg;

    btn.addEventListener('click', () => {
      if (config.isAction) {
        if (config.id === 'clear') {
          this.callbacks.onClear();
        }
      } else {
        this.setActive(config.id as ShapeMode);
        this.callbacks.onSelectMode(config.id as ShapeMode);
      }
    });

    btn.addEventListener('mouseenter', e => {
      this.showTooltip(config.tooltip, e.currentTarget as HTMLElement);
    });

    btn.addEventListener('mouseleave', () => {
      this.hideTooltip();
    });

    this.buttons.set(config.id, btn);
    this.container.appendChild(btn);
  }
```

```
  }

  setActive(mode: ShapeMode): void {
    this.activeMode = mode;
    this.buttons.forEach((btn, id) => {
      if (id === mode) {
        btn.classList.add('is-pressed');
        btn.setAttribute('aria-pressed', 'true');
      } else {
        btn.classList.remove('is-pressed');
        btn.setAttribute('aria-pressed', 'false');
      }
    });
  }

  getActiveMode(): ShapeMode {
    return this.activeMode;
  }

  private showTooltip(text: string, anchorEl: HTMLElement): void {
    this.tooltipEl.textContent = text;
    this.tooltipEl.hidden = false;

    const rect = anchorEl.getBoundingClientRect();
    const tooltipRect = this.tooltipEl.getBoundingClientRect();

    const left = rect.left + rect.width / 2 - tooltipRect.width / 2;
    const top = rect.bottom + 6;

    this.tooltipEl.style.left = `${Math.max(4, left)}px`;
    this.tooltipEl.style.top = `${top}px`;
  }

  private hideTooltip(): void {
    this.tooltipEl.hidden = true;
  }
}
```

src/index.ts (Головний процес Electron, системне меню з радіо-позначками та IPC-синхронізація)

```
import { app, BrowserWindow, Menu, dialog, ipcMain } from 'electron';
import path from 'path';

let mainWindow: BrowserWindow | null = null;

const createMenu = () => {
  const template: Electron.MenuItemConstructorOptions[] = [
    {
      label: 'Файл',
```

```
    submenu: [
      {
        label: 'Очистити полотно',
        accelerator: 'CmdOrCtrl+N',
        click: () => {
          mainWindow?.webContents.send('menu:clear');
        },
      },
      { type: 'separator' },
      {
        label: 'Вихід',
        accelerator: 'CmdOrCtrl+Q',
        click: () => {
          app.quit();
        },
      },
    ],
  },
  {
    label: "Об'єкти",
    submenu: [
      {
        id: 'mode-point',
        type: 'radio',
        checked: true,
        label: 'Крапка',
        accelerator: 'CmdOrCtrl+1',
        click: () => {
          mainWindow?.webContents.send('menu:mode', 'point');
        },
      },
      {
        id: 'mode-line',
        type: 'radio',
        checked: false,
        label: 'Лінія',
        accelerator: 'CmdOrCtrl+2',
        click: () => {
          mainWindow?.webContents.send('menu:mode', 'line');
        },
      },
      {
        id: 'mode-rect',
        type: 'radio',
        checked: false,
        label: 'Прямокутник',
        accelerator: 'CmdOrCtrl+3',
        click: () => {
          mainWindow?.webContents.send('menu:mode', 'rect');
```

```
        },
      },
      {
        id: 'mode-ellipse',
        type: 'radio',
        checked: false,
        label: 'Еліпс',
        accelerator: 'CmdOrCtrl+4',
        click: () => {
          mainWindow?.webContents.send('menu:mode', 'ellipse');
        },
      },
    ],
  },
  {
    label: 'Довідка',
    submenu: [
      {
        label: 'Про програму',
        click: () => {
          if (mainWindow) {
            dialog.showMessageBox(mainWindow, {
              type: 'info',
              title: 'Про програму',
              message:
                'Лабораторна робота №3\nГрафічний редактор об’єктів на TypeScript + Electron',
              detail:
                'Варіант 26 (Присяжний С. О., група ІМ-55)\n\n' +
                '• Статичний масив: 126 об’єктів\n' +
                '• Гумовий слід: суцільна синя лінія\n' +
                '• Прямокутник: по двох протилежних кутах, блакитне заповнення\n' +
                '• Еліпс: від центру до кута охопл. прямокутника, біле заповнення\n' +
                '• Панель інструментів (Toolbar): кнопки з іконками, фіксація стану та підказки\n' +
                '• Позначка режиму: у меню (радіо-позначка) та на панелі інструментів',
            });
          }
        },
      },
    ],
  },
];

  const menu = Menu.buildFromTemplate(template);
  Menu.setApplicationMenu(menu);
};

const createWindow = () => {
  mainWindow = new BrowserWindow({
    width: 1000,
```

```
    height: 700,
    title: 'Графічний редактор — Лабораторна робота №3',
    webPreferences: {
      preload: path.join(process.cwd(), 'dist/preload.js'),
      contextIsolation: true,
      nodeIntegration: false,
    },
  });

  mainWindow.loadFile(path.resolve(process.cwd(), 'index.html'));
  createMenu();
};

ipcMain.on('window:set-title', (_event, title: string) => {
  if (mainWindow && title) {
    mainWindow.setTitle(title);
  }
});

ipcMain.on('mode:changed', (_event, mode: string) => {
  const item = Menu.getApplicationMenu()?.getMenuItemById(`mode-${mode}`);
  if (item) {
    item.checked = true;
  }
});

app.whenReady().then(() => {
  createWindow();

  app.on('activate', () => {
    if (BrowserWindow.getAllWindows().length === 0) {
      createWindow();
    }
  });
});

app.on('window-all-closed', () => {
  if (process.platform !== 'darwin') {
    app.quit();
  }
});
```

src/preload.ts (Міст безпечної IPC-взаємодії контекстів)

```
import { contextBridge, ipcRenderer } from 'electron';

contextBridge.exposeInMainWorld('electronAPI', {
  onSelectMode: (callback: (mode: string) => void) => {
    ipcRenderer.on('menu:mode', (_event, mode: string) => callback(mode));
```

```
  },
  onClear: (callback: () => void) => {
    ipcRenderer.on('menu:clear', () => callback());
  },
  setWindowTitle: (title: string) => {
    ipcRenderer.send('window:set-title', title);
  },
  notifyModeChanged: (mode: string) => {
    ipcRenderer.send('mode:changed', mode);
  },
});
```

src/renderer.ts (Головний контролер Renderer-процесу App, зв'язування Canvas, Toolbar та Редактора)

```
import { ShapeObjectsEditor, type ShapeMode } from './shape_objects_editor.ts';
import { ToolBar } from './toolbar.ts';

class App {
  private canvas: HTMLCanvasElement;
  private ctx: CanvasRenderingContext2D;
  private editor: ShapeObjectsEditor;
  private toolbar: ToolBar;
  private statusCoords: HTMLElement;
  private statusMode: HTMLElement;
  private statusCount: HTMLElement;

  constructor() {
    this.canvas = document.querySelector<HTMLCanvasElement>('#canvas')!;
    this.ctx = this.canvas.getContext('2d')!;
    this.statusCoords = document.querySelector<HTMLElement>('#status-coords')!;
    this.statusMode = document.querySelector<HTMLElement>('#status-mode')!;
    this.statusCount = document.querySelector<HTMLElement>('#status-count')!;

    this.editor = new ShapeObjectsEditor(
      () => this.repaint(),
      title => {
        window.electronAPI?.setWindowTitle(title);
        if (this.editor) {
          this.updateStatusBar();
        }
      },
      mode => {
        this.toolbar?.setActive(mode);
        window.electronAPI?.notifyModeChanged(mode);
      },
    );

    this.toolbar = new ToolBar('#toolbar', '#tooltip', {
```

```
    onSelectMode: mode => {
      this.editor.setMode(mode);
      this.updateStatusBar();
      window.electronAPI?.notifyModeChanged(mode);
    },
    onClear: () => {
      this.editor.clear();
      this.updateStatusBar();
    },
  });

  this.initCanvasSize();
  this.bindEvents();
  this.updateStatusBar();
  this.repaint();
}

private initCanvasSize(): void {
  const resize = () => {
    this.canvas.width = this.canvas.clientWidth;
    this.canvas.height = this.canvas.clientHeight;
    this.repaint();
  };

  window.addEventListener('resize', resize);
  resize();
}

private bindEvents(): void {
  this.canvas.addEventListener('mousedown', e => {
    if (e.button !== 0) return;
    this.editor.onLBdown(e.offsetX, e.offsetY);
  });

  this.canvas.addEventListener('mousemove', e => {
    this.statusCoords.textContent = `X: ${Math.round(e.offsetX)}, Y: ${Math.round(e.offsetY)}`;
    this.editor.onMouseMove(e.offsetX, e.offsetY);
  });

  this.canvas.addEventListener('mouseup', e => {
    if (e.button !== 0) return;
    this.editor.onLBup(e.offsetX, e.offsetY);
    this.updateStatusBar();
  });

  window.electronAPI?.onSelectMode(mode => {
    this.editor.setMode(mode as ShapeMode);
    this.toolbar.setActive(mode as ShapeMode);
    this.updateStatusBar();
```

```
    });

    window.electronAPI?.onClear(() => {
      this.editor.clear();
      this.updateStatusBar();
    });
  }

  private updateStatusBar(): void {
    if (!this.editor) return;
    this.statusMode.textContent = `Режим: ${this.editor.getCurrentModeName()}`;
    this.statusCount.textContent = `Об'єктів: ${this.editor.getShapeCount()}/${this.editor.getMaxShapes()}`;
  }

  private repaint(): void {
    this.editor.onPaint(this.ctx, this.canvas.width, this.canvas.height);
  }
}

new App();
```

src/shape_objects_editor.ts (Клас-координатор буфера 126 фігур та перемикання редакторів)

```
import { Shape } from './shapes/shape.ts';
import { ShapeEditor } from './editors/shape_editor.ts';
import { PointEditor } from './editors/point_editor.ts';
import { LineEditor } from './editors/line_editor.ts';
import { RectEditor } from './editors/rect_editor.ts';
import { EllipseEditor } from './editors/ellipse_editor.ts';

export type ShapeMode = 'point' | 'line' | 'rect' | 'ellipse';

export class ShapeObjectsEditor {
  private readonly maxShapes: number = 126;
  private pcshape: (Shape | null)[] = Array.from({ length: 126 }, () => null);
  private shapeCount: number = 0;

  private currentEditor: ShapeEditor | null = null;
  private currentMode: ShapeMode = 'point';
  private currentModeName: string = 'Крапка';

  constructor(
    private readonly requestRepaint: () => void,
    private readonly onTitleChange: (title: string) => void,
    private readonly onModeChange?: (mode: ShapeMode) => void,
  ) {
    this.startPointEditor();
  }
```

```
addShape(shape: Shape): void {
  if (this.shapeCount < this.maxShapes) {
    this.pcshape[this.shapeCount++] = shape;
  }
}

startPointEditor(): void {
  this.currentEditor = new PointEditor(
    s => this.addShape(s),
    () => this.requestRepaint(),
  );
  this.currentMode = 'point';
  this.currentModeName = 'Крапка';
  this.updateTitle();
  this.onModeChange?.(this.currentMode);
}

startLineEditor(): void {
  this.currentEditor = new LineEditor(
    s => this.addShape(s),
    () => this.requestRepaint(),
  );
  this.currentMode = 'line';
  this.currentModeName = 'Лінія';
  this.updateTitle();
  this.onModeChange?.(this.currentMode);
}

startRectEditor(): void {
  this.currentEditor = new RectEditor(
    s => this.addShape(s),
    () => this.requestRepaint(),
  );
  this.currentMode = 'rect';
  this.currentModeName = 'Прямокутник';
  this.updateTitle();
  this.onModeChange?.(this.currentMode);
}

startEllipseEditor(): void {
  this.currentEditor = new EllipseEditor(
    s => this.addShape(s),
    () => this.requestRepaint(),
  );
  this.currentMode = 'ellipse';
  this.currentModeName = 'Еліпс';
  this.updateTitle();
  this.onModeChange?.(this.currentMode);
}
```

```
setMode(mode: ShapeMode): void {
  switch (mode) {
    case 'point':
      this.startPointEditor();
      break;
    case 'line':
      this.startLineEditor();
      break;
    case 'rect':
      this.startRectEditor();
      break;
    case 'ellipse':
      this.startEllipseEditor();
      break;
  }
}

private updateTitle(): void {
  this.onTitleChange(
    `Графічний редактор — Лабораторна робота №3 (${this.shapeCount}/${this.maxShapes})`,
  );
}

clear(): void {
  this.pcshape.fill(null);
  this.shapeCount = 0;
  this.updateTitle();
  this.requestRepaint();
}

onLBdown(x: number, y: number): void {
  this.currentEditor?.onLBdown(x, y);
}

onMouseMove(x: number, y: number): void {
  this.currentEditor?.onMouseMove(x, y);
}

onLBup(x: number, y: number): void {
  this.currentEditor?.onLBup(x, y);
  this.updateTitle();
}

onPaint(ctx: CanvasRenderingContext2D, width: number, height: number): void {
  ctx.fillStyle = '#ffffff';
  ctx.fillRect(0, 0, width, height);

  for (let i = 0; i < this.shapeCount; i++) {
```

```
      this.pcshape[i]?.show(ctx);
    }

    this.currentEditor?.onPaint(ctx);
  }

  getShapeCount(): number {
    return this.shapeCount;
  }

  getMaxShapes(): number {
    return this.maxShapes;
  }

  getCurrentMode(): ShapeMode {
    return this.currentMode;
  }

  getCurrentModeName(): string {
    return this.currentModeName;
  }
}
```

src/editors/shape_editor.ts (Базовий редактор введення з синім гумовим слідом #2563eb)

```
import { Editor } from './editor.ts';
import { Shape } from '../shapes/shape.ts';

export abstract class ShapeEditor extends Editor {
  protected isDrawing: boolean = false;
  protected xstart: number = 0;
  protected ystart: number = 0;
  protected xend: number = 0;
  protected yend: number = 0;

  protected readonly rubberColor: string = '#2563eb';

  constructor(
    protected readonly onShapeCreated: (shape: Shape) => void,
    protected readonly requestRepaint: () => void,
  ) {
    super();
  }

  onLBdown(x: number, y: number): void {
    this.xstart = this.xend = x;
    this.ystart = this.yend = y;
    this.isDrawing = true;
    this.requestRepaint();
```

```
  }

  onMouseMove(x: number, y: number): void {
    if (!this.isDrawing) return;
    this.xend = x;
    this.yend = y;
    this.requestRepaint();
  }

  onLBup(x: number, y: number): void {
    if (!this.isDrawing) return;
    this.xend = x;
    this.yend = y;
    this.isDrawing = false;
  }
}
```

src/editors/rect_editor.ts (Редактор прямокутників введенням за двома протилежними кутами)

```
import { ShapeEditor } from './shape_editor.ts';
import { RectShape } from '../shapes/rect_shape.ts';

export class RectEditor extends ShapeEditor {
  override onLBup(x: number, y: number): void {
    super.onLBup(x, y);
    this.onShapeCreated(
      new RectShape(this.xstart, this.ystart, this.xend, this.yend),
    );
    this.requestRepaint();
  }

  override onPaint(ctx: CanvasRenderingContext2D): void {
    if (!this.isDrawing) return;
    const left = Math.min(this.xstart, this.xend);
    const top = Math.min(this.ystart, this.yend);
    const width = Math.abs(this.xend - this.xstart);
    const height = Math.abs(this.yend - this.ystart);

    ctx.save();
    ctx.strokeStyle = this.rubberColor;
    ctx.lineWidth = 1;
    ctx.beginPath();
    ctx.strokeRect(left, top, width, height);
    ctx.restore();
  }
}
```

src/editors/ellipse_editor.ts (Редактор еліпсів введенням від центру до кута охоплюючого прямокутника)

```
import { ShapeEditor } from './shape_editor.ts';
import { EllipseShape } from '../shapes/ellipse_shape.ts';

export class EllipseEditor extends ShapeEditor {
  override onLBup(x: number, y: number): void {
    super.onLBup(x, y);
    this.onShapeCreated(
      new EllipseShape(this.xstart, this.ystart, this.xend, this.yend),
    );
    this.requestRepaint();
  }

  override onPaint(ctx: CanvasRenderingContext2D): void {
    if (!this.isDrawing) return;
    const rx = Math.abs(this.xend - this.xstart);
    const ry = Math.abs(this.yend - this.ystart);

    if (rx === 0 && ry === 0) return;

    ctx.save();
    ctx.strokeStyle = this.rubberColor;
    ctx.lineWidth = 1;
    ctx.beginPath();
    ctx.ellipse(this.xstart, this.ystart, rx, ry, 0, 0, Math.PI * 2);
    ctx.stroke();
    ctx.restore();
  }
}
```

src/shapes/rect_shape.ts (Клас прямокутника з чорним контуром та блакитним заповненням)

```
import { Shape } from './shape.ts';

export class RectShape extends Shape {
  show(ctx: CanvasRenderingContext2D): void {
    const left = Math.min(this.xs1, this.xs2);
    const top = Math.min(this.ys1, this.ys2);
    const width = Math.abs(this.xs2 - this.xs1);
    const height = Math.abs(this.ys2 - this.ys1);

    ctx.save();
    ctx.fillStyle = '#38bdf8';
    ctx.strokeStyle = '#000000';
    ctx.lineWidth = 1;
    ctx.beginPath();
    ctx.rect(left, top, width, height);
```

```
    ctx.fill();
    ctx.stroke();
    ctx.restore();
  }
}
```

src/shapes/ellipse_shape.ts (Клас еліпса від центру з чорним контуром та білим заповненням)

```
import { Shape } from './shape.ts';

export class EllipseShape extends Shape {
  show(ctx: CanvasRenderingContext2D): void {
    const rx = Math.abs(this.xs2 - this.xs1);
    const ry = Math.abs(this.ys2 - this.ys1);

    if (rx === 0 && ry === 0) return;

    ctx.save();
    ctx.fillStyle = '#ffffff';
    ctx.strokeStyle = '#000000';
    ctx.lineWidth = 1;
    ctx.beginPath();
    ctx.ellipse(this.xs1, this.ys1, rx, ry, 0, 0, Math.PI * 2);
    ctx.fill();
    ctx.stroke();
    ctx.restore();
  }
}
```

index.html (Розмітка вікна: панель Toolbar, полотно Canvas, статус-бар та елемент Tooltip)

```
<!doctype html>
<html lang="uk">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Графічний редактор</title>
    <link rel="stylesheet" href="src/styles/index.css" />
  </head>
  <body>
    <div class="editor-container">
      <header id="toolbar" class="toolbar"></header>
      <canvas id="canvas"></canvas>
      <footer class="status-bar">
        <span id="status-mode" class="status-item">Режим: Крапка</span>
        <span class="status-separator">|</span>
        <span id="status-count" class="status-item">Об'єктів: 0/126</span>
        <span class="status-separator">|</span>
```

```
      <span id="status-coords" class="status-item">X: 0, Y: 0</span>
      <span class="status-spacer"></span>
      <span class="status-hint"
        >Оберіть фігуру в меню "Об'єкти" або на панелі інструментів</span
      >
    </footer>
  </div>
  <div id="tooltip" class="tooltip" hidden></div>
  <script type="module" src="dist/renderer.js"></script>
 </body>
</html>
```

src/styles/index.css (Стилі оформлення Toolbar, затиснутого стану кнопок та спливаючих підказок)

```
@import './normalize.css';

:root {
 --color-bg: #ffffff;
 --color-status-bg: #f8fafc;
 --color-status-border: #e2e8f0;
 --color-toolbar-bg: #f8fafc;
 --color-toolbar-border: #e2e8f0;
 --color-toolbar-btn-hover: #e2e8f0;
 --color-toolbar-btn-active: #dbeafe;
 --color-toolbar-border-active: #3b82f6;
 --color-text: #1e293b;
 --color-text-muted: #64748b;
 --color-accent: #2563eb;
}

html,
body {
 width: 100%;
 height: 100%;
 margin: 0;
 padding: 0;
 overflow: hidden;
 font-family:
  system-ui,
  -apple-system,
  sans-serif;
 user-select: none;
}

.editor-container {
 display: flex;
 flex-direction: column;
 width: 100vw;
 height: 100vh;
```

```
}

.toolbar {
  height: 38px;
  background-color: var(--color-toolbar-bg);
  border-bottom: 1px solid var(--color-toolbar-border);
  display: flex;
  align-items: center;
  padding: 0 8px;
  gap: 4px;
  box-sizing: border-box;
  flex-shrink: 0;
}

.toolbar-btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 30px;
  height: 30px;
  border: 1px solid transparent;
  border-radius: 4px;
  background: transparent;
  color: var(--color-text);
  cursor: pointer;
  padding: 0;
}

.toolbar-btn:hover {
  background-color: var(--color-toolbar-btn-hover);
  border-color: #cbd5e1;
}

.toolbar-btn.is-pressed {
  background-color: var(--color-toolbar-btn-active);
  border-color: var(--color-toolbar-border-active);
  color: var(--color-accent);
  box-shadow: inset 0 1px 2px rgba(0, 0, 0, 0.12);
}

.toolbar-separator {
  width: 1px;
  height: 20px;
  background-color: #cbd5e1;
  margin: 0 4px;
}

#canvas {
  flex: 1;
```

```
  width: 100%;
  height: calc(100% - 66px);
  background-color: var(--color-bg);
  cursor: crosshair;
  display: block;
}

.status-bar {
  height: 28px;
  background-color: var(--color-status-bg);
  border-top: 1px solid var(--color-status-border);
  display: flex;
  align-items: center;
  padding: 0 14px;
  font-size: 12px;
  color: var(--color-text);
  gap: 10px;
  box-sizing: border-box;
  flex-shrink: 0;
}

.status-item {
  font-weight: 500;
}

.status-separator {
  color: var(--color-status-border);
}

.status-spacer {
  flex: 1;
}

.status-hint {
  color: var(--color-text-muted);
  font-size: 11px;
}

.tooltip {
  position: absolute;
  z-index: 1000;
  background-color: #1e293b;
  color: #ffffff;
  padding: 4px 8px;
  border-radius: 4px;
  font-size: 11px;
  pointer-events: none;
  box-shadow: 0 2px 6px rgba(0, 0, 0, 0.2);
  white-space: nowrap;
```

```
}
```

Ілюстрації роботи програми (скріншоти):

У ході тестування графічного редактора Лабораторної роботи №3 було проведено всебічну перевірку нового функціоналу панелі інструментів, механізму підказок, фіксації активного стану та коректності відображення оновлених за варіантом 26 геометричних фігур.



Рис. 3. Головне вікно графічного редактора у початковому стані: відображення Toolbar з активною кнопкою «Крапка», роздільником та кнопкою очищення полотна



Рис. 4. Демонстрація механізму спливаючих підказок (Tooltips): підказка при наведенні курсору на кнопку «Прямокутник» із зазначенням гарячої клавіші



Рис. 5. Введення точкових примітивів (Point): створення точок на полотні та оновлення лічильника в рядку стану (7/126)



Рис. 6. Введення лінійних об'єктів (Line): демонстрація суцільного синього «гумового» сліду (#2563eb) згідно з варіантом 26

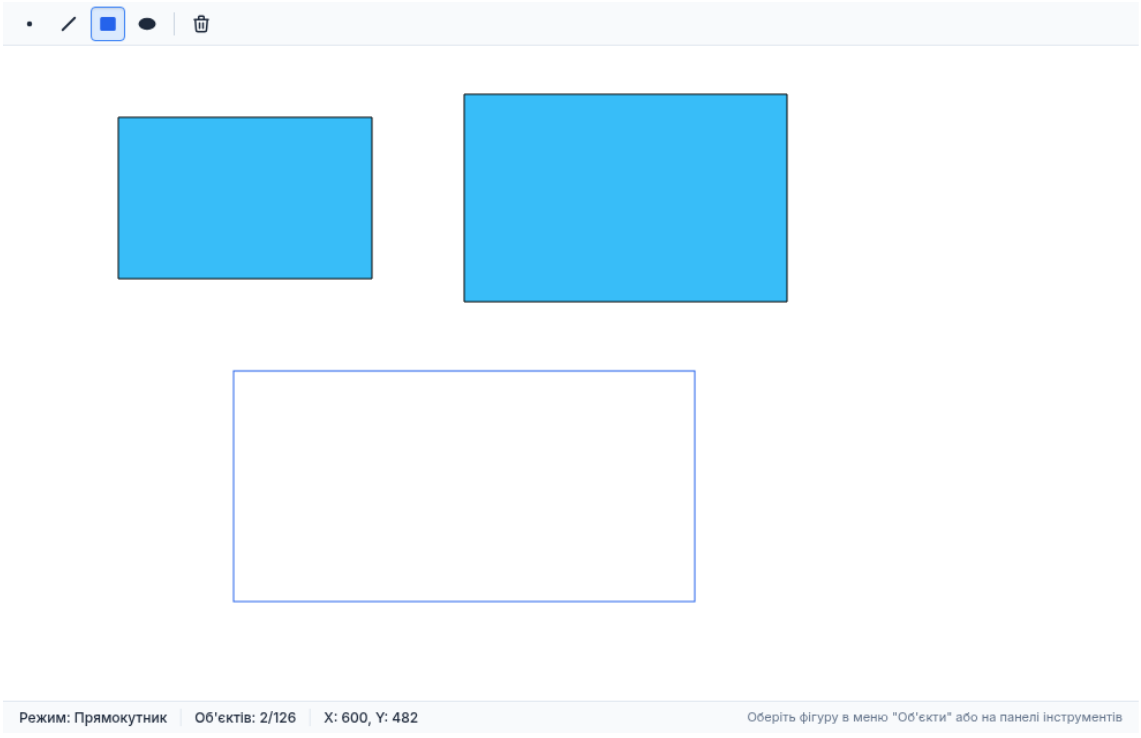


Рис. 7. Введення прямокутників (Rect) по двох протилежних кутах: блакитне заповнення (#38bdf8), чорний контур та синій гумовий слід

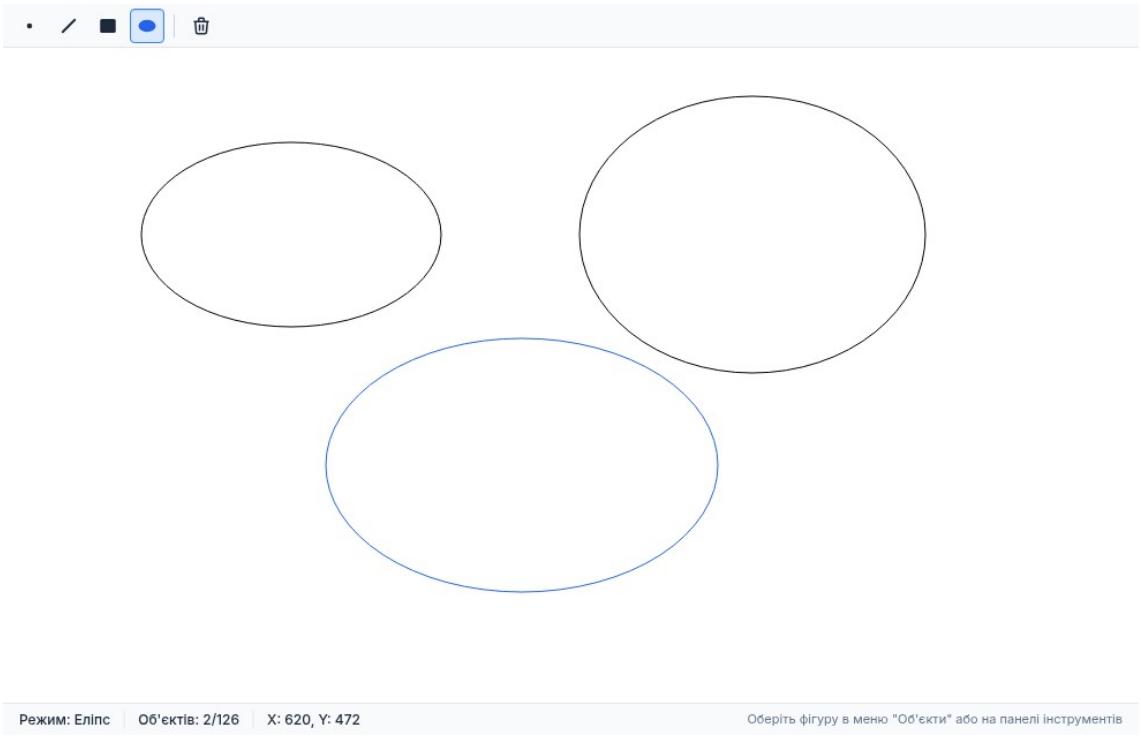


Рис. 8. Введення еліпсів (Ellipse) від центру до кута: біле заповнення (#ffffff), чорний контур та синій гумовий контур введення

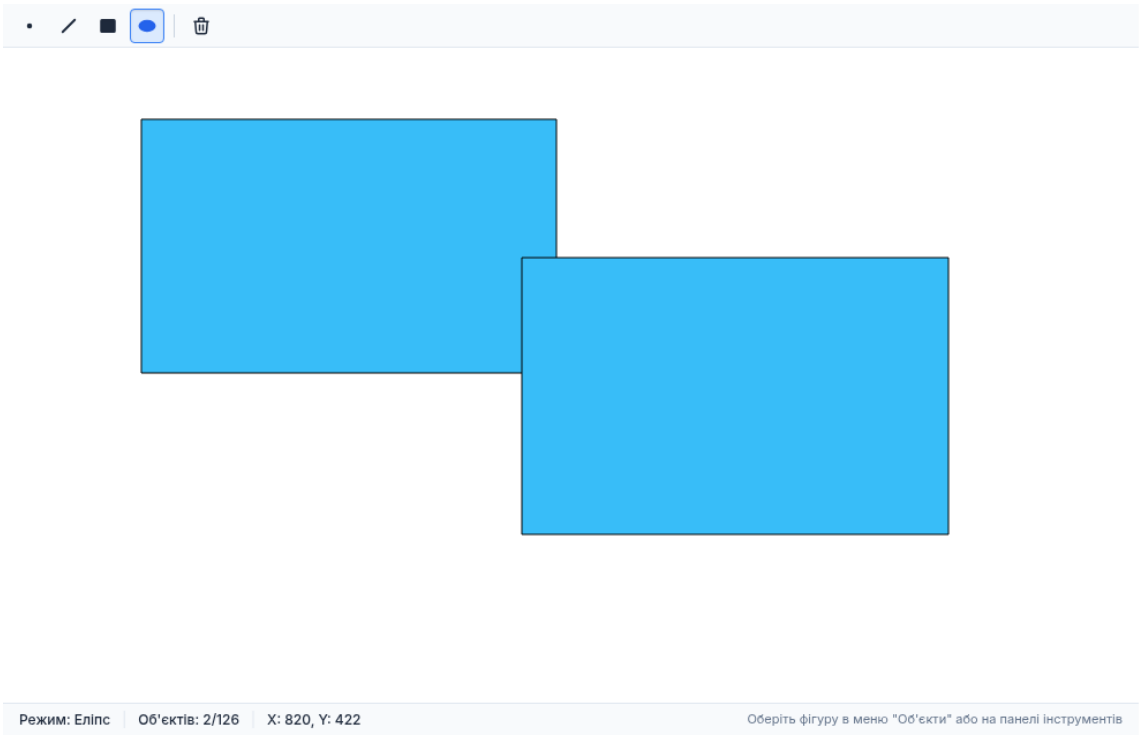


Рис. 9. Загальна композиція фігур: наочна демонстрація взаємного перекриття точок, ліній, блакитних прямокутників та непрозорих білих еліпсів

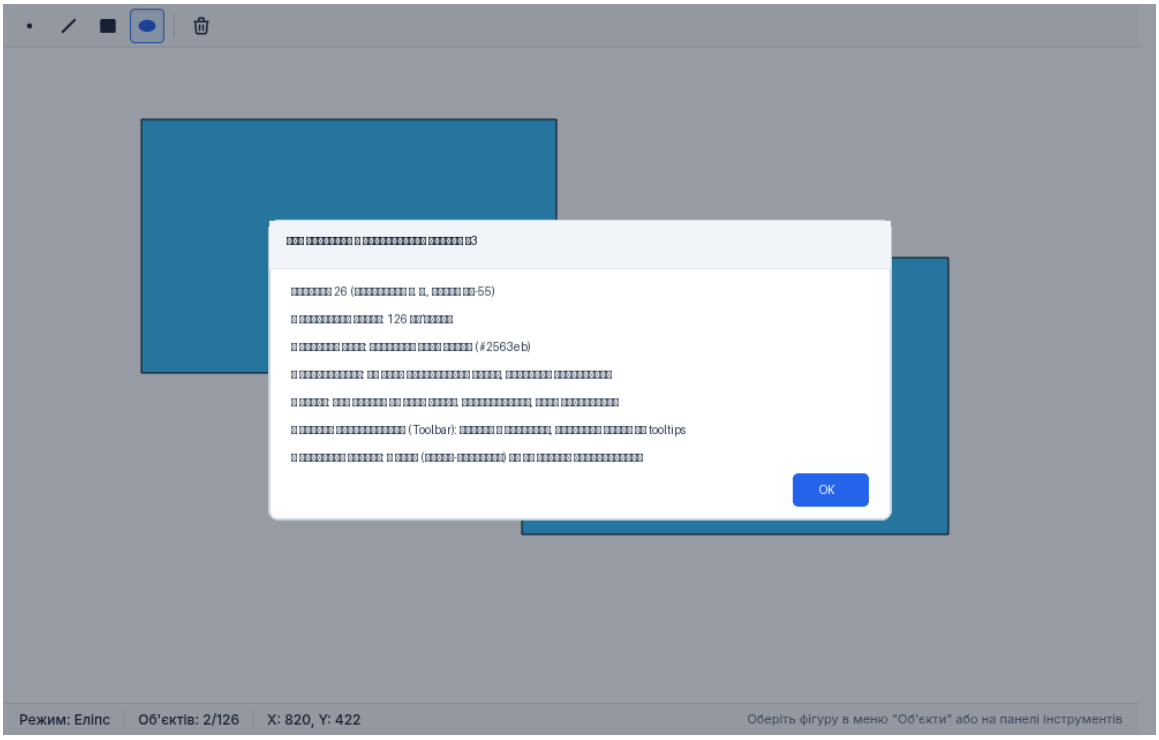


Рис. 10. Інформаційне вікно «Про програму» з детальним описом розрахункових характеристик індивідуального варіанта 26

Відповіді на контрольні запитання:

Нижче наведено вичерпні відповіді на контрольні запитання до лабораторної роботи №3 (стор. 68 методичного посібника):

1. У чому полягає об'єктна орієнтованість ОС Windows?

Об'єктна орієнтованість операційної системи Windows базується на уніфікованій концепції вікон та обміні повідомленнями:
• Кожен візуальний компонент графічного інтерфейсу (головне вікно застосунку, діалогове вікно, панель інструментів Toolbar, кнопка, поле введення, статус-бар тощо) розглядається як вікно, ідентифіковане дескриптором HWND.
• Кожне вікно є екземпляром зареєстрованого в системі класу вікна (WNDCLASS / WNDCLASSEX) і пов'язане з власною віконною процедурою (WndProc), яка інкапсулює реакцію на події.
• Усі елементи підпорядковані чіткій ієрархії: головне вікно є батьківським (parent), а розміщені на його поверхні кнопки та панелі — дочірніми (child).
• Взаємодія між системою і компонентами відбувається шляхом надсилання стандартизованих повідомлень (наприклад, WM_CREATE, WM_SIZE, WM_COMMAND, WM_NOTIFY, WM_PAINT).

2. Обробку яких повідомлень потрібно виконувати у програмі Лаб3?

У графічному редакторі Лабораторної роботи №3 виконується обробка наступних ключових віконних повідомлень:
• WM_CREATE (OnCreate) — створення дочірнього вікна Toolbar за допомогою CreateToolbarEx або CreateWindowEx, додавання кнопок та завантаження іконок.
• WM_SIZE (OnSize) — реакція на зміну геометрії головного вікна та корегування ширини панелі Toolbar викликом MoveWindow.
• WM_COMMAND (OnCommand) — реакція на вибір пунктів меню або натискання кнопок Toolbar за спільними командними ідентифікаторами.
• WM_NOTIFY (OnNotify) — обробка сповіщення TTN_NEEDTEXT від елементів Toolbar для формування тексту спливаючих підказок (Tooltips).
• WM_LBUTTONDOWN, WM_MOUSEMOVE, WM_LBUTTONUP — обробка маніпулятора миша для введення координат фігур та малювання гумового сліду.
• WM_PAINT (OnPaint) — відмальовування вмісту полотна та збережених об'єктів з буфера pcshape.
• OnInitMenuPopup — динамічна перевірка та встановлення радіо-позначки активного типу фігури у випадаючому меню «Об'єкти».

3. Що таке абстрактний клас і скільки їх у цій програмі?

Абстрактний клас — це базовий клас, що визначає загальний інтерфейс та спільний стан для похідних класів певної ієрархії, але не дозволяє створювати власні екземпляри напряму (в C++ містить щонайменше одну чисто віртуальну функцію з = 0, у TypeScript декларується за допомогою ключового слова abstract).
У розробленій програмі реалізовано 2 абстрактні класи:
1. Shape — абстрактний базовий клас геометричних об'єктів. Інкапсулює змінні координат (xs1, ys1, xs2, ys2) та оголошує чисто віртуальний метод show(ctx).
2. Editor (та ShapeEditor) — абстрактний базовий клас контролерів введення, що визначає інтерфейс обробки миші (onLBdown, onMouseMove, onLBup, onPaint) та реалізує спільну логіку синього гумового сліду.

4. Як забезпечити відповідність пунктів меню і кнопок Toolbar?

Для забезпечення взаємної відповідності пунктів меню і кнопок Toolbar використовуються єдині командні ідентифікатори та централізована синхронізація стану:
• Кнопці Toolbar та відповідному підпункту меню присвоюється однаковий ID (наприклад, ID_TOOL_RECT та IDM_RECT). При їхній активації віконна процедура отримує одне й те саме повідомлення WM_COMMAND.
• Фіксація стану: при виборі фігури вікну Toolbar надсилається повідомлення TB_PRESSBUTTON для візуального «втискання» активної кнопки, а попередньо активна кнопка відтискається.
• Позначка в меню: за допомогою CheckMenuRadioItem (або обробника OnInitMenuPopup) встановлюється позначка (радіо-маркер) біля активного пункту меню

«Об'єкти».
• У нашій реалізації на TypeScript/Electron синхронізація здійснюється через спільний тип ShapeMode, двосторонні IPC-повідомлення (menu:mode та mode:changed) та метод setActive() класу ToolBar.

5. Як запрограмувати показ власних зображень на кнопках Toolbar?

• У класичному WinAPI для показу власних зображень створюється ресурс BITMAP, який містить послідовність кадрів кнопок. Вікну Toolbar надсилається повідомлення TB_ADDBITMAP зі структурою TBADDBITMAP, або ресурс передається напряму у CreateToolbarEx. Для кожної кнопки у структурі TBBUTTON поле iBitmap вказує цілочисельний індекс відповідного зображення у растрі.
• У нашому проєкті на базі TypeScript/Electron показ власних зображень реалізовано за допомогою векторних SVG-піктограм у конфігурації класу ToolBar. Векторні піктограми забезпечують бездоганну чіткість без розмиття на дисплеях будь-якої роздільної здатності.

6. Як створити власні зображення кнопок і де вони зберігаються?

• У середовищі Visual Studio C++ власні растрові зображення створюються через Solution Explorer: контекстне меню проекту $\to$ Add $\to$ Resource $\to$ Bitmap. Зображення малюються вбудованим графічним редактором як горизонтальна растрова смуга розміром (кількість_кнопок * 24) x 24 пікселів. Файл зберігається у папці проекту (наприклад, res/toolbar.bmp) та підключається до скрипту ресурсів (.rc) із числовим ідентифікатором у resource.h.
• У веб-застосунку піктограми зберігаються як масштабовані векторні шаблони SVG або файли в структурі assets, що забезпечує апаратне рендерення та стилізацію через CSS.

7. Як запрограмувати текст підказок (tooltips)?

• У WinAPI для реалізації підказок:
1. Під час створення Toolbar додається віконний стиль TBSTYLE_TOOLTIPS.
2. У віконній процедурі WndProc реалізується обробник повідомлення WM_NOTIFY.
3. При отриманні сповіщення з кодом TTN_NEEDTEXT параметру lParam надається тип LPTOOLTIPTEXT. За ідентифікатором lpttt->hdr.idFrom визначається кнопка, і в буфер lpttt->szText копіюється відповідний текстовий рядок (lstrcpy(lpttt->szText, L"Крапка")).
• У нашому проєкті на TypeScript клас ToolBar інкапсулює події mouseenter / mouseleave на елементах кнопок, зчитує текст підказки та динамічно вираховує екранні координати плаваючого DOM-елемента #tooltip за допомогою getBoundingClientRect().

Висновки:

У ході виконання лабораторної роботи №3 було успішно спроєктовано та програмно реалізовано повнофункціональний графічний інтерфейс користувача (GUI) для

векторного графічного редактора на основі принципів об'єктно-орієнтованого програмування:

1. Реалізовано новий інтерфейсний компонент ToolBar, що інкапсулює створення кнопок вибору інструментів, генерацію векторних піктограм, механізм спливаючих підказок (Tooltips) та керування фіксацією натиснутого стану (pressed state).
2. Забезпечено повну двосторонню синхронізацію між кнопками панелі інструментів, гарячими клавішами (Ctrl+1..4) та підпунктами системного меню «Об'єкти», де реалізовано радіо-індикацію активного режиму введення згідно з варіантом Ж = 26.
3. Відповідно до індивідуального варіанта 26 успішно модифіковано графічні примітиви та контролери:

- розмір статичного буфера об'єктів збільшено до 126 покажчиків;
- колір інтерактивного «гумового» сліду змінено на суцільний синій (#2563eb);
- алгоритм прямокутника налаштовано на введення по двох кутах із блакитною заливкою (#38bdf8);
- алгоритм еліпса налаштовано на введення від центру до одного з кутів охоплюючого прямокутника з білим заповненням (#ffffff).

4. Завдяки чіткому дотриманню принципів поліморфізму та інкапсуляції, розширення системи новим компонентом ToolBar та зміна поведінки окремих фігур відбулися без порушення загального контракту координатора ShapeObjectsEditor, що підтверджує гнучкість розробленої об'єктної архітектури.